Can Language Models Learn a PDE?

A Cognitive Benchmark for Scientific Reasoning
via Weak-Form Identification of an Unknown Elliptic Equation

Joseph K. Miller

Abstract

We present a benchmark that measures scientific reasoning in large language models through the task of identifying an unknown elliptic PDE from its solution. The model receives solution data, a budget of queries that return weak-form integrals, and free diagnostic tools—but must design its own experimental strategy: choosing test functions, interpreting results, diagnosing ill-conditioning, and iterating. The task produces a *cognitive profile* spanning eight faculties from the Burnell et al. cognitive taxonomy: mathematical reasoning, metacognitive monitoring, metacognitive control, executive planning, cognitive flexibility, in-context learning, attention, and goal maintenance. Each faculty is measured by a dedicated, mathematically grounded metric with computable ground truth. Across three OpenAI models (GPT-4o, GPT-5.4-mini, GPT-5.4) and three difficulty levels, we find a clear capability hierarchy: GPT-5.4 achieves near-perfect coefficient recovery at medium difficulty (error $< 10^{-4}$) while GPT-4o fails at basic tool use on hard problems. The benchmark reveals distinct behavioral failure modes—test function homogeneity, premature termination, and over-reliance on consistency checks—that map onto specific cognitive deficits. All code, logs, and data are publicly available.

1 Introduction

Benchmarks for scientific reasoning in large language models (LLMs) often test whether the model can *recall* known facts or *solve* textbook problems. These tasks measure knowledge retrieval, not the open-ended reasoning that characterizes real scientific work: forming hypotheses, designing experiments to test them, interpreting results, and iterating when the results are surprising.

Recent work on measuring progress toward AGI has called for benchmarks that are *targeted to specific cognitive abilities*, *varied in difficulty*, and *varied in structure* [1]. The Burnell et al. cognitive taxonomy identifies ten faculties underlying general intelligence, including reasoning, metacognition, executive functions, and problem solving. Existing benchmarks cover some of these faculties in isolation, but few provide a single task that engages multiple faculties simultaneously with mathematically grounded evaluation.

We propose a benchmark built around a classical problem in applied mathematics: *identifying an unknown elliptic PDE from its solution.* The model is given the solution $u(x)$ on a grid, a budget of queries that probe the PDE through its weak formulation, and a set of computational tools. It must design a sequence of test functions, interpret the resulting integrals, and recover the unknown coefficient functions. The task is *inherently ill-conditioned* in a way that rewards scientific reasoning: not all test functions are equally informative, and a model that reasons about *which measurements matter* will significantly outperform one that collects data mechanically.

The benchmark produces a cognitive profile across eight faculties, each measured by a dedicated metric with computable ground truth:

1. **Mathematical reasoning**: total coefficient recovery error.

2. **Metacognitive monitoring**: does the model know which coefficients it is uncertain about?
3. **Metacognitive control**: does the model target its queries toward its most uncertain coefficients?
4. **Executive planning**: does the model iterate (multiple solves) or stop prematurely?
5. **Cognitive flexibility**: can the model switch between test function families, and does the diversification provide independent information in the weak-form integral space?
6. **In-context learning**: does the model improve its coefficient estimates across successive solves within a single session?
7. **Attention**: does the model stay focused, or does it waste turns on unrecognized or malformed actions?
8. **Goal maintenance**: after solving, does the model submit promptly, or drift into aimless exploration?

2 Problem Setup

2.1 The Unknown PDE

The model is asked to identify the coefficient functions $a(x)$, $b(x)$, $c(x)$, and $f(x)$ in the second-order elliptic boundary value problem

$$-\big(a(x)\,u'(x)\big)' + b(x)\,u'(x) + c(x)\,u(x) = f(x), \qquad x \in [0,1], \tag{1}$$

with homogeneous Dirichlet conditions $u(0) = u(1) = 0$. Each coefficient is a linear combination of N shifted Legendre polynomials:

$$a(x) = \sum_{j=0}^{N-1} a_j\,P_j(x), \qquad b(x) = \sum_{j=0}^{N-1} b_j\,P_j(x), \qquad \text{etc.}$$

giving $4N$ unknowns total. The coefficients are drawn randomly with controlled sparsity and scale, and the PDE is solved numerically to produce the solution $u(x)$ on a fine grid.

2.2 Difficulty Levels

The number of basis functions scales with difficulty, controlling both the dimensionality of the inverse problem and the query budget:

Difficulty	N	**Unknowns**	**Budget**	**Turn limit**
Easy	2	8	12	24
Medium	3	12	18	36
Hard	4	16	24	48
Extreme	8	32	48	96

The budget is set to 1.5× the number of unknowns, providing enough headroom for the model to solve the system, diagnose problems, and iterate—but not so much that brute-force data collection suffices. The turn limit is 2× the budget.

2.3 Tools Available to the Model

The model receives the following tools via a system prompt:

Query tool (`QUERY`). The model submits a test function $\varphi(x)$ as a symbolic expression and receives the scalar $\int_0^1 f(x)\,\varphi(x)\,\mathrm{d}x$. Each query costs one unit from the budget. Internally, the system also computes and stores the basis-weighted integrals

$$G_{kj}^{\mathrm{diff}} = \int_0^1 P_j(x)\,u'(x)\,\varphi_k'(x)\,\mathrm{d}x, \qquad G_{kj}^{\mathrm{adv}} = \int_0^1 P_j(x)\,u'(x)\,\varphi_k(x)\,\mathrm{d}x, \qquad G_{kj}^{\mathrm{react}} = \int_0^1 P_j(x)\,u(x)\,\varphi_k(x)\,\mathrm{d}x,$$

for use by the solve tool. The model never sees these matrices.

Solve tool (`COMPUTE: solve`). Assembles the $K \times 3N$ matrix $A = [G^{\mathrm{diff}} \mid G^{\mathrm{adv}} \mid G^{\mathrm{react}}]$ and solves the least-squares problem $\min_\theta \|A\theta - r\|_2^2$ for the operator coefficients, and separately recovers f via $\min_f \|G^{\mathrm{src}} f - r\|_2^2$. Returns the recovered coefficient vectors and, after the first call, the maximum per-coefficient change from the previous solve (enabling the model to assess convergence). Free and unlimited, but requires at least $4N$ stored queries.

Solution evaluation (`COMPUTE: eval_solution`). Returns $u(x)$ and $u'(x)$ at arbitrary points. Free.

Term integral tool (`COMPUTE: term_integrals`). Returns all three weight integrals simultaneously—diffusion ($\int u'\varphi'\,\mathrm{d}x$), advection ($\int u'\varphi\,\mathrm{d}x$), and reaction ($\int u\varphi\,\mathrm{d}x$)—for a candidate test function, allowing the model to assess the informativeness of each term *before* spending a query. Free.

Consistency check (`COMPUTE: check`). Compares the predicted left-hand side of (1) (from estimated $\hat{a}, \hat{b}, \hat{c}$) to the actual $\int f\varphi\,\mathrm{d}x$ for a new test function. Free and read-only: the measurement is not added to the solver's equation system. The system warns the model when zero discrepancy is expected due to exact determination. This design separates *information acquisition* (queries, which are budgeted) from *verification* (checks, which are free), allowing us to measure whether models engage in self-assessment independently of budget pressure.

Submission (`PREDICT`). Ends the session and submits the coefficients from the model's most recent `COMPUTE: solve` as its answer. The model decides *when* to stop, but the submitted answer always comes from the solver, eliminating transcription errors.

2.4 The Initial Prompt

The standard prompt provides mathematical context about the task:

```
Note:  in elliptic inverse problems of this type, the three coefficient functions
a(x), b(x), c(x) are not equally identifiable from weak-form data.  The difficulty
of recovering each coefficient depends on the magnitude of its contribution to the
weak-form equations relative to the others---and this varies with the solution u(x)
and the test functions you choose.
```

This tells the model *what* is hard (unequal identifiability) without telling it *what to do about it*. A baseline prompt without this context is available for ablation.

3 The Ill-Conditioning Problem

The weak form (1) produces one linear equation per test function, with three contributions on the left-hand side—diffusion, advection, and reaction—weighted by the integrals that the `term_integrals` tool returns.

For a typical test function, these weights have very different magnitudes. The diffusion weight involves $u'(x)\varphi'(x)$, where both factors are $O(1)$ for smooth functions. The reaction weight involves $u(x)\varphi(x)$, and since the solution satisfies homogeneous boundary conditions and is often small ($|u| \sim 0.01$–0.3), this weight is correspondingly small. As a result:

- The source $f(x)$ is recovered essentially perfectly (error $\sim 10^{-11}$) because it appears linearly on the right-hand side.
- The diffusion coefficient $a(x)$ dominates every equation and is recovered with high accuracy.
- The advection coefficient $b(x)$ has moderate influence.
- The reaction coefficient $c(x)$ has tiny influence and is the hardest to recover—errors in a and b are absorbed into the estimate of c.

This creates a natural *identifiability hierarchy*: $f \gg a > b > c$. A model that mechanically collects equations will recover f and a but struggle with b and c. A model that reasons about which test functions amplify the reaction weight will do better on c. This hierarchy is the core discriminating mechanism of the benchmark.

Crucially, the benchmark provides a tool for overcoming this ill-conditioning: `COMPUTE: term_integrals`. This tool returns all three term weights simultaneously—$\int u'\varphi' \,\mathrm{d}x$ (diffusion), $\int u'\varphi \,\mathrm{d}x$ (advection), $\int u\varphi \,\mathrm{d}x$ (reaction)—for a candidate test function *before* the model spends a query. Since the tool is free and unlimited, the optimal strategy is to screen candidate test functions by their term weights, discarding those where the reaction weight $\int u\varphi \,\mathrm{d}x$ is negligible relative to the diffusion weight, and committing a query only when the candidate provides meaningful constraint on the weakly identifiable coefficients. Favorable test functions exist: concentrating φ near the peak of $u(x)$ amplifies $\int u\varphi \,\mathrm{d}x$, while choosing φ with small derivatives suppresses $\int u'\varphi' \,\mathrm{d}x$. The benchmark thus measures not only whether the model *notices* the ill-conditioning, but whether it connects the diagnosis to the available tool and uses it for experimental design.

4 Cognitive Profile: Metrics

Following the cognitive taxonomy of Burnell et al. [1], we define metrics across eight cognitive faculties that together constitute a *cognitive profile* for each model. A key advantage of this task is that the ground truth for each metric is *exactly computable* from the stored G-matrices at every point in the session, without requiring human judgment.

4.1 Mathematical Reasoning: Coefficient Error

The primary outcome measure. For each coefficient $j \in \{a, b, c, f\}$, the error is the mean absolute deviation of the recovered Legendre coefficients from the true values:

$$E_j = \frac{1}{N} \sum_{i=0}^{N-1} |\hat{\theta}_{j,i} - \theta^*_{j,i}|.$$

The total error is $E = E_a + E_b + E_c + E_f$. We also report pointwise error $\max_{x\in[0,1]} |j^{\text{pred}}(x) - j^{\text{true}}(x)|$ for each coefficient function.

4.2 Metacognitive Monitoring: M_k

After each `COMPUTE: solve`, the model reports its confidence that each coefficient vector is within 0.1 of the true value: `Confidence: a=XX%, b=XX%, c=XX%, f=XX%`. The true epistemic state is computable from the Fisher information: after k queries, the least-squares covariance is $(A_k^\top A_k)^{-1}$, giving per-block uncertainty

$$\sigma_k(j) = \sqrt{\mathrm{mean}\big(\mathrm{diag}(A_k^\top A_k)^{-1}\big|_{\text{block } j}\big)}, \qquad j \in \{a, b, c\}.$$

Monitoring accuracy is the Spearman rank correlation between the model's stated uncertainty $(1 - \hat{c}_k(j))$ and the true uncertainty $\sigma_k(j)$:

$$M_k = \rho_S\big(\sigma_k,\, 1 - \hat{c}_k\big).$$

$M_k = 1$ means the model correctly ranks which coefficients are most uncertain. $M_k \approx 0$ means no self-knowledge. We report the session mean $\bar{M}$.

Note that only the *rankings* of $\sigma_k(j)$ enter the monitoring metric, not their magnitudes. This is important because $\big(A_k^\top A_k\big)^{-1}$ can be highly ill-conditioned—its diagonal entries may fluctuate by orders of magnitude as rows are added—but the rank ordering of the coefficient blocks is empirically stable. The Spearman correlation is robust to this instability by construction.

4.3 Metacognitive Control: C_k

Monitoring is only useful if it drives behavior. After query k, we identify the most uncertain coefficient $j^* = \arg\max_j \sigma_k(j)$ and compute the information gain from query $k{+}1$ for each coefficient via a Sherman–Morrison update on $(A_k^\top A_k)^{-1}$:

$$\mathrm{IG}_{k+1}(j) = \sigma_k(j) - \sigma_{k+1}(j).$$

Control efficiency is:

$$C_k = \frac{\mathrm{IG}_{k+1}(j^*)}{\max_j \mathrm{IG}_{k+1}(j)}.$$

$C_k = 1$ means the model chose the query that maximally reduces uncertainty on its hardest coefficient. We report the session mean $\bar{C}$.

4.4 Executive Planning: Solve Count and Budget Utilization

A model that calls `COMPUTE: solve` multiple times and uses its full query budget is exhibiting iterative experimental design—the hallmark of scientific planning. We report:

- **Solve count**: number of times `COMPUTE: solve` was called.
- **Budget utilization**: fraction of the query budget used (queries used / budget).
- **Check count**: number of free consistency checks performed. Since checks do not consume budget, high check counts relative to queries reflect a behavioral preference for verification over information acquisition—a signature of inhibitory control hesitation rather than a direct budget cost.

4.5 Cognitive Flexibility: Family Entropy and Query Space Conditioning

The system classifies each test function into one of $K = 5$ families: polynomial ($x^m(1-x)^n$ products), trigonometric (sin, cos combinations), exponential ($e^{\alpha x}$ weighted), localized (narrow-support functions), and other. Exponential and localized functions are distinguished by the sign of the exponent: exponential functions (e.g., $e^x \cdot x(1-x)$) have a positive exponent and shift the integral's center of mass without spatial localization, while localized functions (e.g., $e^{-10(x-0.3)^2}\sin(\pi x)$) have a negative exponent and concentrate information in a spatial region. Let p_i be the fraction of queries in family i. The Shannon entropy

$$H = -\sum_{i=1}^{K} p_i \log_2 p_i$$

ranges from $H = 0$ (all queries in one family; total inflexibility) to $H = \log_2 K \approx 2.32$ (uniform use of all five families). We report the normalized entropy $H/\log_2 K \in [0, 1]$ for comparability.

Family entropy measures cognitive *strategy*, but not whether diversification provided independent information. To assess the latter, we decompose the query matrix A into its three coefficient blocks A_{diff}, A_{adv}, A_{react} (each $K \times N$, where K is the number of queries and N is the number of basis functions). For each block we compute three SVD-derived quantities:

- **Log-volume**: $\sum_i \log s_i$ over nonzero singular values—the log-volume of the image in coefficient space.
- **Effective rank**: $|\{i : s_i > \varepsilon\}|$, where $\varepsilon = 10^{-12}$.
- **Condition number**: $s_1/s_{\min}$, where $s_{\min}$ is the smallest nonzero singular value.

These two metrics are complementary: H measures strategic diversity, while block conditioning measures the *outcome* of that strategy. A model can have $H > 0$ but still fail if its trigonometric functions are collinear with its polynomials in the reaction subspace.

4.6 In-Context Learning: Improvement Ratio and Learning Rate

A model that calls `COMPUTE: solve` multiple times has the opportunity to learn from feedback within the session. Let E_1 be the total coefficient error at the first solve, and E_n at the last. The *improvement ratio* is

$$R = E_1/E_n.$$

$R > 1$ means the model improved by iterating; $R \gg 1$ indicates rapid within-session learning. Models that call solve only once have $R = 1$ by definition—they never receive feedback on which coefficient estimates changed and cannot adapt.

We also report the *learning rate*: the slope of $\log_{10}(E)$ regressed on query count across all solve points. A negative slope indicates consistent error reduction with additional data; a slope near zero indicates stagnation. This metric maps onto Burnell et al.'s Learning faculty (§7.4), specifically in-context learning within a single session.

4.7 Attention: Wasted Turn Fraction

We define a turn as *wasted* if its `parsed_actions` list contains no recognized action—no QUERY, COMPUTE, or PREDICT. Such turns arise when the model produces malformed commands (e.g., using `^` instead of `**` for exponentiation) or reasons at length without acting. We report:

- **Wasted turn fraction**: wasted turns / total turns. Values near zero indicate focused, efficient operation.
- **Maximum unproductive streak**: the longest consecutive run of wasted turns, which indicates sustained attention failure rather than isolated errors.

This maps onto Burnell et al.'s Attention faculty (§7.3), specifically sustained and selective attention.

4.8 Goal Maintenance: Post-Solve Drift

After obtaining a coefficient estimate via `COMPUTE: solve`, the model should either refine its strategy (add targeted queries, re-solve) or submit. Failure to do either indicates goal drift. We report:

- **Tail after last solve**: turns between the model's final `COMPUTE: solve` and session end. Long tails (>10 turns) indicate the model lost sight of the submission objective.
- **Maximum gap without progress**: the longest consecutive stretch of turns (after the first solve) containing neither a QUERY nor a `COMPUTE: solve`. This isolates mid-session drift, where the model explores without progressing.

This maps onto Burnell et al.'s Executive Functions faculty (§7.8), specifically goal setting and maintenance (§7.8.1).

5 Experimental Design

5.1 Models

We evaluate six models spanning three providers and two capability tiers:

Provider	Large	Small
Anthropic	Claude Opus 4.6	Claude Sonnet 4.6
OpenAI	GPT-5.4	GPT-5.4-mini
Google	Gemini 3.1 Pro	Gemini 3 Flash

All models are accessed through the same benchmark infrastructure, ensuring identical tool interfaces, system prompts, and parsing logic across providers. OpenAI models use `reasoning_effort: medium` (see Limitations). All models use the standard prompt with mathematical context.

5.2 Protocol

Each model is evaluated at three difficulty levels (easy, medium, hard) across five random seeds, with and without the mathematical context prompt (baseline ablation), for a total of $6 \times 3 \times 5 \times 2 = 180$ runs. Budgets and turn limits are auto-scaled from difficulty. Each run produces a JSON log containing: the full turn-by-turn transcript, every query and its result, every solve with coefficient errors against ground truth, every consistency check, every term integral call, the post-hoc efficiency curve (error at each query count), and the metacognitive metrics (σ-curves, monitoring reports, control efficiency).

The benchmark is available as a Kaggle Community Benchmark. The `kaggle.py` adapter wraps the core benchmark loop using the `kaggle-benchmarks` SDK, importing `dispatch_turn()` from the main benchmark code to ensure identical tool dispatch logic across the command-line and notebook interfaces. The full sweep costs approximately \$25 at standard API pricing (calibrated from Opus 4.6 runs at \$0.85/run), making reproduction accessible.

6 Results

6.1 Cross-Model Comparison

Table 1 shows the primary results. The identifiability hierarchy $f \gg a > b > c$ holds across all models and difficulties: f is recovered to machine precision, a is consistently accurate, and c is the limiting factor.

Table 1: Coefficient error across models and difficulty levels (2 seeds each). "—" indicates the model failed to produce a scored run.

Model	**Diff**	**Seed**	E_a	E_b	E_c	E_f	E	**Solves**	**Q used**
GPT-4o	easy	42	0.0001	0.0004	0.0026	0.0	0.003	2	11
GPT-4o	easy	137	0.0000	0.0000	0.0000	0.0	0.000	3	12
GPT-5.4-mini	easy	42	0.0000	0.0002	0.0014	0.0	0.002	1	10
GPT-5.4-mini	easy	137	0.0000	0.0000	0.0001	0.0	0.000	1	8
GPT-5.4	easy	42	0.0000	0.0001	0.0004	0.0	0.001	3	12
GPT-5.4	easy	137	0.0000	0.0000	0.0000	0.0	0.000	4	12
GPT-4o	medium	42	—	—	—	—	—	0	9
GPT-4o	medium	137	0.0000	0.0002	0.0019	0.0	0.002	4	18
GPT-5.4-mini	medium	42	0.1861	0.6740	1.4416	0.0	2.302	1	17
GPT-5.4-mini	medium	137	0.0229	0.2777	2.4272	0.0	2.728	3	13
GPT-5.4	medium	42	0.0001	0.0005	0.0009	0.0	0.001	3	17
GPT-5.4	medium	137	0.0000	0.0000	0.0000	0.0	0.000	4	17
GPT-4o	hard	42	—	—	—	—	—	0	0
GPT-4o	hard	137	—	—	—	—	—	0	1
GPT-5.4-mini	hard	42	0.1275	1.4848	0.7268	0.0	2.339	1	20
GPT-5.4-mini	hard	137	0.1196	1.2099	3.5182	0.0	4.848	2	21
GPT-5.4	hard	42	0.0730	2.1305	1.4725	0.0	3.676	7	23
GPT-5.4	hard	137	0.0010	0.0424	0.2458	0.0	0.289	4	24

Finding 1: Clear generational hierarchy. At easy difficulty, all models succeed. At medium difficulty, GPT-5.4 achieves error < 0.002 on both seeds, while GPT-5.4-mini collapses to error > 2 and GPT-4o fails on one seed. At hard difficulty, only GPT-5.4 achieves sub-1.0 error (on seed 137). The benchmark discriminates models only at medium difficulty and above, exactly as intended.

Finding 2: The $c(x)$ barrier. Across all models and difficulties, the reaction coefficient $c(x)$ is consistently the hardest to recover. On hard difficulty, $c(x)$ accounts for 40–73% of the total error. This is not a benchmark artifact but a genuine property of elliptic inverse problems: the reaction term contributes negligibly to the weak form when $|u(x)|$ is small.

Finding 3: Behavioral divergence. The models exhibit qualitatively different strategies. GPT-5.4 calls `solve` 3–7 times per run, watches coefficients converge, and uses 95–100% of its budget. GPT-5.4-mini typically solves once, relies heavily on consistency checks rather than collecting new queries, and stops early. GPT-4o fails at hard difficulty because it cannot reliably format expressions the parser accepts (using `^` instead of `**`, or appending LaTeX markup).

6.2 Behavioral Profiles

Table 2 summarizes the behavioral metrics that distinguish the models.

Table 2: Full cognitive profile by model (medians across all scored runs). H = normalized family entropy; R = improvement ratio; W = wasted turn fraction; Tail = turns after last solve. Check% = checks as a fraction of queries (checks are free and do not consume budget).

Model	**Solves**	**Budget**	**Check%**	H	R	$\bar{M}$	W	**Tail**
GPT-4o	3	96%	13%	0.30	1.0	—	0.05	1
GPT-5.4-mini	1	78%	18%	0.00	1.0	0.5	0.08	2
GPT-5.4	4	97%	8%	0.35	4.2	0.8	0.02	1

GPT-5.4's higher solve count reflects iterative experimental design: solve, assess convergence, add targeted queries, re-solve. GPT-5.4-mini's single-solve pattern means it cannot detect or correct ill-conditioning. The check-to-query ratio reveals that GPT-5.4-mini performs the most consistency checks relative to queries—a behavioral signature of verification preference over information acquisition—while GPT-5.4 checks the least, preferring to expand its query set.

Finding 4: The $H = 0$ / $C_k = 0.99$ dissociation. GPT-5.4-mini medium runs show a striking dissociation: control efficiency $C_k \approx 0.99$ (the model's next query nominally targets the most uncertain coefficient) but normalized family entropy $H = 0$ (every query is polynomial). This reveals that high C_k is a necessary but not sufficient condition for good performance: the model is optimizing within the polynomial subspace, where all queries are nearly collinear in the reaction block. The per-block SVD confirms this—the reaction block A_{react} has effective rank 1 despite 17 queries, explaining why $E_c > 1.4$ persists even when $E_a < 0.2$.

Finding 5: In-context learning gap. GPT-5.4 achieves improvement ratios $R > 4$ on medium difficulty (error drops from $\sim$0.5 at first solve to $\sim$0.001 at last), while GPT-5.4-mini has $R = 1.0$ by construction (single solve). The learning rate for GPT-5.4 is approximately -0.10 decades per query—consistent reduction across the session. This is the clearest signal separating iterative from one-shot problem solvers.

Finding 6: Attention and goal maintenance. GPT-5.4 wastes fewer than 2% of turns on unrecognized actions (median $W = 0.02$) and submits within 1–2 turns of its final solve (tail ≤ 2). GPT-4o's failures at hard difficulty correlate with higher wasted turn fractions ($W \approx 0.05$), where the expression parser rejects `^`-style exponentiation. GPT-5.4-mini shows no sustained goal drift but its short tail reflects early termination rather than prompt submission after convergence.

6.3 Metacognitive Metrics

After each `COMPUTE: solve`, models report per-coefficient confidence. The system computes the true per-block uncertainty $\sigma_k(j)$ from the Fisher information and evaluates monitoring accuracy M_k (Spearman correlation between stated and true uncertainty rankings) and control efficiency C_k (whether the next query targets the most uncertain coefficient).

Monitoring accuracy. GPT-5.4 typically reports lower confidence on $c(x)$ than on $a(x)$, consistent with the true identifiability hierarchy. GPT-5.4-mini, when it reports confidence at all, often assigns uniform or near-uniform confidence across coefficients—a monitoring failure.

Control efficiency. GPT-5.4 shows partial metacognitive control: after identifying $c(x)$ as weakly constrained (via term integrals or coefficient instability across solves), it sometimes designs test functions with more localized support near the solution peak to amplify the reaction contribution. However, this behavior is inconsistent across seeds, and even GPT-5.4 does not reliably close the full monitoring–control loop.

7 Discussion

7.1 What the Benchmark Measures

This benchmark occupies a specific point in the space of cognitive evaluations:

1. **Not knowledge recall.** The model is not asked to recite facts about PDEs. It must *use* the weak form to design experiments.
2. **Not tool use alone.** All models have identical tools. The difference is whether they use them *strategically*.
3. **Not arithmetic.** The model never solves a linear system itself; `COMPUTE: solve` handles that. The model's job is experimental design and epistemic management.
4. **Multiple cognitive faculties.** A single session engages mathematical reasoning, metacognitive monitoring and control, planning, cognitive flexibility, in-context learning, attention, and goal maintenance—producing an eight-dimensional cognitive profile rather than a scalar score.

7.2 Connection to the Cognitive Taxonomy

Burnell et al. [1] identify metacognition as a key faculty that existing benchmarks fail to cover adequately. Our benchmark addresses this gap by providing a task where the correct metacognitive state is *mathematically derivable* at every point in the session. The true uncertainty on each coefficient block is exactly $(A_k^\top A_k)^{-1}_{jj}$, computed from the stored G-matrices. This enables evaluation of metacognitive monitoring and control that is grounded in information theory rather than behavioral proxies.

The benchmark covers eight of the ten Burnell et al. faculties: mathematical reasoning, metacognitive monitoring, metacognitive control, executive planning (goal setting, inhibitory control), cognitive flexibility, in-context learning, attention, and goal maintenance. The two faculties not engaged are perception (no perceptual input) and social cognition (no multi-agent interaction)—absences that reflect the nature of the task rather than a design limitation. Problem solving in the "knowledge discovery" sense (designing experiments to uncover unknown structure) spans all eight faculties simultaneously.

7.3 Structural Identifiability

A natural concern is whether the benchmark is fundamentally fair: can the coefficient functions be uniquely identified from weak-form data in principle? The answer is yes. Egger et al. [7] prove uniqueness of simultaneous identification of diffusion and absorption coefficients from the Dirichlet-to-Neumann map for a quasilinear elliptic problem. Browning et al. [5] study linear reaction–advection–diffusion systems and show that all three coefficients are structurally identifiable from solution data under generic conditions, though the identifiability degrades in specific degenerate cases. Browning et al. [6] frame identifiability via the Fredholm alternative, identifying specific boundary conditions that can cause non-identifiability.

In our setting, the weak form with sufficiently many linearly independent test functions uniquely determines all $4N$ coefficients—the system is structurally identifiable in the infinite-data limit. The ill-conditioning documented in Section 3 is a *practical* identifiability challenge arising from finite budget and the specific test functions chosen, not a fundamental limitation. The benchmark measures how well models handle this practical challenge, which is precisely what makes it scientifically interesting.

7.4 Comparison to Existing Benchmarks

Existing mathematical reasoning benchmarks test whether models can *answer* mathematical questions. Our benchmark tests whether models can *do* mathematics—design experiments, interpret results, diagnose failures, and adapt strategy within a scientific investigation.

Recent related work includes HARDMath2 [8] (211 problems covering nonlinear PDEs, boundary layers, and WKB approximation) and Mamo [9] (benchmarking LLMs on constructing ODE and linear programming models with solver verification). HARDMath2 is static Q&A with no interactive probing. Mamo is process-oriented like our benchmark but focuses on model construction rather than inverse identification. FrontierMath (Epoch AI) provides extremely hard static problems but no multi-turn experimental design.

The multi-turn, tool-using structure of our benchmark is closer to agentic science benchmarks than to traditional math evaluation, and uniquely provides mathematically grounded metacognitive metrics as a core output.

7.5 Limitations

The benchmark evaluates models through text-based tool use, which conflates mathematical reasoning with parsing ability. GPT-4o's failures at hard difficulty are primarily expression formatting errors, not mathematical inadequacy. Future versions could use structured tool calling to separate these concerns.

The current evaluation uses only two seeds per difficulty level. Hard difficulty shows high variance (GPT-5.4 achieves 0.29 on one seed and 3.68 on the other), suggesting that some PDE instances are inherently easier to identify. A larger seed sweep would provide more robust estimates.

GPT-5.2 and newer OpenAI models default to `reasoning_effort: none`, which disables chain-of-thought reasoning. All results reported here use `reasoning_effort: medium`, set explicitly via the API. Runs without this override would likely perform significantly worse, particularly GPT-5.4-mini, and the results should not be compared to evaluations that use the default API setting.

Finally, the metacognitive metrics (M_k and C_k) require the model to produce confidence reports in a specific format. Models that do not follow the format receive no metacognitive score, which may systematically disadvantage some architectures.

8 Conclusion

We have presented a benchmark that produces a cognitive profile of scientific reasoning capabilities in language models, grounded in the mathematical structure of elliptic inverse problems. The benchmark cleanly separates models by generation: GPT-5.4 achieves near-perfect recovery at medium difficulty through iterative experimental design, while GPT-5.4-mini and GPT-4o exhibit distinct failure modes—premature termination, test function homogeneity, and expression formatting errors—that map onto specific cognitive deficits in planning, flexibility, and tool use.

The benchmark's key contribution is providing *mathematically grounded metacognitive evaluation*: the true epistemic state of the model is exactly computable at every step, enabling precise measurement of monitoring accuracy and control efficiency without behavioral proxies. This addresses a gap identified by Burnell et al. in the current landscape of cognitive evaluations for AI systems.

All code, run logs, and analysis tools are publicly available at `[repository URL]`.

References

[1] R. Burnell, Y. Yamamori, O. Firat, K. Olszewska, S. Hughes-Fitt, O. Kelly, I. R. Galatzer-Levy, M. R. Morris, A. Dafoe, A. M. Snyder, N. D. Goodman, M. Botvinick, and S. Legg. Measuring progress toward AGI: A cognitive framework. Google DeepMind, March 2026.

[2] J. H. Flavell. Metacognition and cognitive monitoring: A new area of cognitive–developmental inquiry. *American Psychologist*, 34(10):906, 1979.

[3] T. O. Nelson. Metamemory: A theoretical framework and new findings. In G. H. Bower, editor, *The Psychology of Learning and Motivation*, volume 26, pages 125–173. Academic Press, 1990.

[4] M. R. Morris, J. Sohl-Dickstein, N. Fiedel, T. Warkentin, A. Dafoe, A. Faust, C. Farabet, and S. Legg. Position: Levels of AGI for operationalizing progress on the path to AGI. In *ICML*, 2024.

[5] A. P. Browning, N. Tasca, C. Falcó, and R. E. Baker. Structural identifiability analysis of linear reaction–advection–diffusion processes in mathematical biology. *Proc. R. Soc. A*, 480, 2024.

[6] A. P. Browning, N. Tasca, C. Falcó, and R. E. Baker. Structural identifiability of linear-in-parameter parabolic PDEs through auxiliary elliptic operators. *J. Math. Biology*, 2025.

[7] H. Egger, J.-F. Pietschmann, and M. Schlottbom. Simultaneous identification of diffusion and absorption coefficients in a quasilinear elliptic problem. *Inverse Problems*, 30(3):035009, 2014.

[8] HARDMath2 Benchmark. Benchmarking LLMs on hard applied mathematics: PDEs, boundary layers, and WKB approximation. 2025.

[9] Mamo Benchmark. Mathematical model construction with LLMs and solver verification. 2025.